LAPORAN PRAKTIKUM

STRUKTUR DATA

“Stack”

disusun oleh: Dennis hauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Struktur Data, dengan Judul "Stack"

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 19 April 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 2

1.1 Latar Belakang 2

1.2 Tujuan 2

1.3 Manfaat 3

BAB II PEMBAHASAN 4

2.1 Pengertian Stack 4

2.2 Implementasi Pada Java 4

2.2.1 Nilai Maksimum_2511533020 4

2.2.2 SiswaStack_2511533020 6

2.2.3 stackArray_2511533020 7

2.2.4 stackArrayDriver_2511533020 8

2.2.5 StackPostfix_2511533020 9

BAB III PENUTUP 11

3.1 Kesimpulan 11

3.2 Saran 11

3.4 Penutup 11

DAFTAR PUSTAKA 11

TUGAS PRAKTIKUM 11

- KELAS ADT 13

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, struktur data yang menerapkan prinsip LIFO (Last In First Out) dikenal dengan nama Stack atau tumpukan. Prinsip ini analog dengan tumpukan piring: piring yang terakhir diletakkan di atas akan menjadi yang pertama diambil. Stack banyak digunakan dalam berbagai aplikasi seperti evaluasi ekspresi matematika (postfix), undo-redo pada editor teks, pelacakan pemanggilan fungsi (call stack), serta manajemen memori. Pada praktikum kali ini, Stack dipelajari sebagai kelanjutan dari materi struktur data dinamis setelah ArrayList.

Berbeda dengan ArrayList yang bersifat sekuensial dan bisa diakses di sembarang posisi, Stack hanya mengizinkan operasi pada satu ujung saja (atas/top). Operasi utama pada Stack adalah push (menambah elemen ke atas), pop (mengambil dan menghapus elemen dari atas), serta peek (melihat elemen atas tanpa menghapus). Karena keterbatasan aksesnya, Stack sangat cocok untuk masalah yang membutuhkan pemrosesan secara terbalik atau berurutan mundur.

Latar belakang pemilihan topik ini adalah karena Stack merupakan struktur data fundamental yang wajib dikuasai sebelum mempelajari rekursi, parsing ekspresi, atau algoritma pencarian seperti DFS (Depth First Search). Praktikum ini menggunakan bahasa Java dengan implementasi bervariasi, mulai dari penggunaan kelas Stack bawaan Java, implementasi Stack menggunakan ArrayList, hingga implementasi Stack menggunakan array statis. Melalui kelima program yang diberikan, mahasiswa diharapkan mampu memahami dan membedakan berbagai cara implementasi Stack serta mengaplikasikannya dalam kasus nyata seperti evaluasi postfix dan pencarian nilai maksimum.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami konsep dasar Stack dan prinsip LIFO (Last In First Out) melalui implementasi dalam bahasa Java. Mahasiswa diharapkan mampu mengoperasikan method-method utama Stack yaitu push, pop, peek, isEmpty, serta mampu membedakan implementasi Stack menggunakan kelas bawaan Java (java.util.Stack), menggunakan ArrayList, dan menggunakan array statis. Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan dalam menyelesaikan permasalahan sederhana dengan

Stack, seperti mencari nilai maksimum dalam Stack tanpa merusak urutan asli, serta mengevaluasi ekspresi aritmatika dalam notasi postfix.

Selanjutnya, melalui kode program yang diberikan (mulai dari NilaiMaksimum_2511533020, SiswaStack_2511533020, stackArray_2511533020, hingga StackPostfix_2511533020), mahasiswa dapat mengamati bagaimana Stack digunakan untuk menyimpan objek buatan sendiri (misal data siswa), bagaimana menangani kondisi overflow dan underflow, serta bagaimana Stack berperan dalam proses komputasi ekspresi. Pada akhir praktikum, mahasiswa diharapkan dapat memilih implementasi Stack yang tepat sesuai dengan kebutuhan program.mempraktikkan cara menampilkan seluruh isi ArrayList menggunakan perulangan, baik dengan for biasa maupun for-each.

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah penguasaan terhadap struktur data Stack yang sangat berguna dalam pengembangan perangkat lunak sehari-hari. Dengan memahami Stack, mahasiswa dapat membuat fitur undo/redo pada aplikasi, mengelola navigasi halaman browser (tombol back/forward), serta memeriksa keseimbangan tanda kurung dalam ekspresi matematika. Implementasi Stack menggunakan ArrayList maupun array statis juga memberikan wawasan tentang trade-off antara kemudahan penggunaan dan efisiensi memori.

Selain itu, praktikum ini memberikan manfaat langsung dalam hal kemampuan debugging dan penalaran algoritmik. Contohnya pada program NilaiMaksimum_2511533020, mahasiswa belajar bagaimana mempertahankan isi Stack asli saat melakukan pencarian nilai maksimum dengan bantuan Stack cadangan. Pada program StackPostfix_2511533020, mahasiswa melihat langsung penerapan Stack dalam evaluasi ekspresi yang umum digunakan dalam compiler dan kalkulator. Semua ini menjadi bekal penting untuk mata kuliah lanjutan seperti Algoritma dan Pemrograman, serta Struktur Data Lanjut.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Stack (tumpukan) adalah struktur data linear yang mengikuti aturan LIFO (Last In First Out), artinya elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Operasi dasar pada Stack meliputi push (menambah elemen di puncak), pop (menghapus dan mengembalikan elemen puncak), peek atau top (melihat elemen puncak tanpa menghapus), dan isEmpty (memeriksa apakah Stack kosong). Stack dapat diimplementasikan menggunakan array (statis atau dinamis) maupun menggunakan kelas koleksi seperti ArrayList atau LinkedList yang sudah disediakan Java.

Keuntungan menggunakan Stack adalah kesederhanaan dan efisiensi waktu untuk operasi push/pop yang berjalan O(1) jika menggunakan array atau ArrayList dengan akses indeks terakhir. Namun, implementasi dengan array statis memiliki keterbatasan ukuran maksimal, sehingga perlu menangani kondisi overflow. Sementara implementasi dengan ArrayList atau Stack bawaan Java bersifat dinamis dan lebih fleksibel. Dalam praktikum ini, kelima program menunjukkan berbagai pendekatan implementasi Stack.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 Nilai Maksimum_2511533020

```java
package pekan3_2511533020;
import java.util.*;
public class NilaiMaksimum_2511533020 {
    public static int max_3020(Stack<Integer> s_3020) {
        Stack<Integer> backup_3020 = new Stack<Integer>();
        int maxValue_3020 = s_3020.pop();
        backup_3020.push(maxValue_3020) ;
        while (!s_3020.isEmpty()) {
            int next_3020 = s_3020.pop();
            backup_3020.push (next_3020);
            maxValue_3020 = Math.max(maxValue_3020, next_3020);
        }
        while (!backup_3020.isEmpty()) {
            s_3020.push(backup_3020.pop());
        }
        return maxValue_3020;
    }
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Stack<Integer> s_3020 = new Stack<Integer>();
        s_3020.push(70);
        s_3020.push(12);
        s_3020.push(20);
        System.out.println("isi stack" + s_3020);
        System.out.println("Stack Teratas " + s_3020.peek());
        System.out.println("Nilai maksimum" + max_3020(s_3020));

    }

}
```

Program ini menunjukkan cara mencari nilai tertinggi dalam sebuah Stack bertipe Integer tanpa merusak urutan asli stack. Method max_3020 menerima parameter Stack<Integer> s_3020. Pertama, program mengambil elemen teratas menggunakan pop() dan menyimpannya sebagai nilai maksimum sementara, sekaligus menyimpan elemen tersebut ke stack cadangan (backup_3020).

Selanjutnya, selama stack asli tidak kosong, program terus mengambil elemen, membandingkan dengan nilai maksimum, dan memindahkan setiap elemen ke stack cadangan. Setelah semua elemen diproses, program mengembalikan semua elemen dari stack cadangan ke stack asli dengan perulangan push.

Dengan cara ini, stack asli kembali ke kondisi semula. Di method main, dibuat stack berisi angka 70, 12, 20, lalu dicetak isi stack, elemen teratas (peek), dan hasil nilai maksimum. Output yang dihasilkan: isi stack[70, 12, 20], Stack Teratas 20, Nilai maksimum 70.

2.2.2 SiswaStack_2511533020

```java
package pekan3_2511533020;
import java.util.*;


class Siswa_2511533020 {
    String nama_3020;
    int nim_3020;
    public Siswa_2511533020(String nama_3020, int nim_3020) {
        this.nama_3020 = nama_3020;
        this.nim_3020 = nim_3020;
    }
    public String toString() {
        return "Nim: " + nim_3020 + ", Nama: " + nama_3020;
    }
}

public class SiswaStack_2511533020 {
    private ArrayList<Siswa_2511533020> stack;



    public SiswaStack_2511533020() {
        stack = new ArrayList<>();

    }


    public void push(Siswa_2511533020 mhs) {
        stack.add(mhs);

    }
    public Siswa_2511533020 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);

        }
        return null;
    }
```

```java

    public Siswa_2511533020 peek() {
        if (!isEmpty()) {
            return stack.get((stack.size()) -1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();

    }

    public void tampilkanSiswa_2511533020() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        SiswaStack_2511533020 studentStack = new SiswaStack_2511533020();

        Siswa_2511533020 mhs1 = new Siswa_2511533020("Ali", 1);
        Siswa_2511533020 mhs2 = new Siswa_2511533020("Boby", 2);
        Siswa_2511533020 mhs3 = new Siswa_2511533020("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Siswa di dalam stack");
        studentStack.tampilkanSiswa_2511533020();

        System.out.println("Siswa Teratas " + studentStack.peek());
        System.out.println("Mengeluarkan Siswa Teratas Dari Stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop :");
        studentStack.tampilkanSiswa_2511533020();
```

Program ini mendemonstrasikan implementasi Stack menggunakan ArrayList sebagai penyimpan internal, dan Stack tersebut digunakan untuk menyimpan objek buatan sendiri yaitu Siswa_2511533020 (dengan atribut nama dan nim). Kelas SiswaStack_2511533020 memiliki atribut private ArrayList<Siswa_2511533020> stack. Konstruktor menginisialisasi stack sebagai ArrayList kosong. Method push menambahkan objek siswa ke akhir ArrayList (puncak stack).

Method pop menghapus dan mengembalikan elemen terakhir (index size()-1). Method peek mengembalikan elemen terakhir tanpa menghapus. Method isEmpty mengecek apakah ArrayList kosong. Method tampilkanSiswa_2511533020 mencetak semua siswa dari puncak

ke dasar (perulangan dari indeks terakhir ke 0). Di method main, dibuat tiga objek siswa (Ali, Boby, Charles) kemudian dipush ke stack. Program mencetak isi stack (dari atas ke bawah), lalu melakukan peek dan pop, lalu mencetak isi setelah pop. Output menunjukkan bahwa siswa Charles (yang terakhir push) menjadi yang pertama pop.

2.2.3 stackArray_2511533020

```java
package pekan3_2511533020;

public class stackArray_2511533020 {
    static final int MAX_3020 = 1000;
    int top_3020;
    int a_3020[] = new int [MAX_3020];
    boolean isEmpty_3020()
    {
        return (top_3020 < 0);

    }
    stackArray_2511533020()
    {
        top_3020 = -1;

    }
    boolean push(int x_3020)
    {
        if (top_3020 >= (MAX_3020 -1 )) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
    a_3020[++top_3020] = x_3020;
            System.out.println(x_3020 + "dimasukan dalam stack");
            return true;
        }
    }
    int pop_3020()
    {
        if (top_3020 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }

        else {
            int x_3020 = a_3020 [top_3020--];
            return x_3020;
```

```java

        }
    }
        int peek_3020()
        {
            if (top_3020 < 0) {
                System.out.println("Stack Underflow");
                return 0;
            }
            else {
                int x_3020  = a_3020[top_3020];
                return x_3020;
        }
        }
        void print_3020 () {
            for(int i_3020 = top_3020; i_3020 > -1; i_3020--) {
                System.out.print(" "+ a_3020[i_3020]);
            }
        }

}
```

Kelas ini mengimplementasikan Stack menggunakan array statis berukuran tetap (MAX_3020 = 1000). Atribut top_3020 menunjukkan indeks elemen teratas, diinisialisasi -1 (kosong). Method push(int x_3020) mengecek apakah stack penuh (top >= MAX_3020-1). Jika penuh, muncul pesan "Stack Overflow". Jika tidak, top ditambah, lalu nilai dimasukkan ke

array. Method pop_3020 mengecek underflow (top < 0). Jika kosong, pesan "Stack Underflow" dan return 0. Jika tidak, nilai pada indeks top diambil lalu top dikurangi.

Method peek_3020 mirip dengan pop namun tidak mengubah top. Method print_3020 mencetak semua elemen dari atas ke bawah. Kelas ini tidak memiliki method main sendiri, melainkan diuji oleh driver terpisah (kode 2.2.2).

2.2.4 stackArrayDriver_2511533020

```java
package pekan3_2511533020;
import java.util.*;
public class stackArrayDriver_2511533020 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        stackArray_2511533020 s_3020 = new stackArray_2511533020();
            s_3020.push(10);
            s_3020.push(20);
            s_3020.push(30);
            System.out.println(s_3020.pop_3020() + " dikeluarkan dari stack");
            System.out.println("Elemen teratas adalah :" + s_3020.peek_3020());
            System.out.println("Elemen pada stack :");
            s_3020.print_3020();

    }

}
```

Driver ini menguji kelas stackArray_2511533020. Objek s_3020 dibuat, kemudian dilakukan push nilai 10, 20, 30. Selanjutnya dipanggil pop_3020 yang mengeluarkan 30 (karena stack LIFO), lalu dicetak nilai yang dikeluarkan.

Method peek_3020 menampilkan elemen teratas sekarang yaitu 20. Terakhir, print_3020 mencetak sisa elemen stack (20 dan 10) dengan urutan dari atas ke bawah. Output yang dihasilkan: 30 dimasukan dalam stack, 20 dimasukan dalam stack, 10 dimasukan dalam stack, 30 dikeluarkan dari stack, Elemen teratas adalah :20, Elemen pada stack : 20 10. Program ini menunjukkan implementasi stack dengan array statis dan penanganan overflow/underflow sederhana.

2.2.5 StackPostfix_2511533020

```java
package pekan3_2511533020;

import java.util.Scanner;

public class StackPostfix_2511533020 {
        public static int postfixEvaluate(String expression) {
            Stack<Integer> s_3020 = new Stack<Integer>() ;
            Scanner input_3020 = new Scanner(expression);
            while (input_3020.hasNext()) {
                if (input_3020.hasNextInt()) {
                    s_3020.push(input_3020.nextInt());
                } else {
                    String operator_3020 = input_3020.next();
                    int operand2_3020 = s_3020.pop();
                    int operand1_3020 = s_3020.pop();
                    if (operator_3020.equals("+")) {
                        s_3020.push(operand1_3020 + operand2_3020);
                    } else if (operator_3020.equals("-")) {
                        s_3020.push(operand1_3020 - operand2_3020);
                    } else if (operator_3020.equals("*")) {
                        s_3020.push(operand1_3020 * operand2_3020);
                    } else {
                        s_3020.push(operand1_3020 / operand2_3020);

                    }

                }
            }
            input_3020.close();
            return s_3020.pop();
        }
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        System.out.println("hasil postfix= "+postfixEvaluate("5 2 4 * + 7 -"));

    }

}
```

Program ini mengevaluasi ekspresi aritmatika dalam notasi postfix (Reverse Polish Notation) menggunakan Stack bawaan Java. Method postfixEvaluate(String expression) menerima string ekspresi postfix, misalnya "5 2 4 * + 7 -". Scanner digunakan untuk membaca token (angka atau operator). Jika token adalah angka, dilakukan push ke stack. Jika token adalah operator, dua operand di-pop (operand2 adalah yang pertama keluar, operand1 kedua), lalu hasil operasi di-push kembali. Operator yang didukung: +, -, *, /.

Setelah semua token diproses, hasil akhir berada di puncak stack dan dikembalikan. Pada method main, ekspresi "5 2 4 * + 7 -" dievaluasi. Prosesnya: push 5,2,4; lalu operator * → 2*4=8 push; operator + → 5+8=13 push; push 7; operator - → 13-7=6. Output: hasil postfix= 6. Program ini adalah contoh nyata penggunaan Stack dalam komputasi ekspresi.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Stack yang telah dilakukan, dapat disimpulkan bahwa Stack merupakan struktur data dengan prinsip LIFO yang dapat diimplementasikan menggunakan berbagai cara: menggunakan kelas Stack bawaan Java, menggunakan ArrayList, maupun menggunakan array statis.

\Kelima program yang diberikan berhasil menunjukkan operasi dasar push, pop, peek, serta penerapan Stack dalam studi kasus seperti mencari nilai maksimum tanpa merusak data, menyimpan objek siswa, menangani kondisi overflow/underflow, dan mengevaluasi ekspresi postfix. Pemahaman tentang Stack sangat penting karena menjadi fondasi untuk topik lanjutan seperti rekursi, parsing, dan algoritma graf (DFS).

3.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar implementasi Stack menggunakan array statis (stackArray_2511533020) ditambahkan kemampuan untuk mengubah ukuran array secara dinamis (resize) agar tidak mudah overflow. Pada program NilaiMaksimum_2511533020, sebaiknya ditambahkan penanganan jika stack kosong. Program SiswaStack_2511533020 dapat dilengkapi dengan method untuk mencari siswa berdasarkan nim.

Selain itu, praktikum berikutnya dapat memperluas penggunaan Stack untuk konversi infix ke postfix atau untuk memeriksa keseimbangan tanda kurung pada ekspresi matematika. Seluruh program sudah memenuhi aturan penamaan dengan akhiran NIM 3020 pada atribut dan variabel lokal yang relevan.

3.4 Penutup

Demikian laporan praktikum Struktur Data dengan judul Stack ini disusun. Laporan ini mencakup penjelasan teoritis, implementasi kode, serta analisis dari lima program yang telah dibuat. Semoga laporan ini bermanfaat untuk memahami konsep Stack dan penerapannya dalam bahasa pemrograman Java.

Penulis menyadari masih terdapat kekurangan, oleh karena itu kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang.

DAFTAR PUSTAKA

[1] Oracle, "Stack (Java Platform SE 17)," OpenJDK. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Stack.html (diakses: 19 April 2026).

[2] D. I. Gunawan Hts et al., Struktur Data dan Algoritma: Pondasi Komputasi Modern. Teori dan Praktik dengan Java. Indonesia: Penerbit Teknosain, 2025.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] W3Schools, "Java Stack," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_stack.asp (diakses: 19 April 2026).

TUGAS PRAKTIKUM

- **KELAS DRIVER:**

```java
package pekan3;

import java.util.*;

public class Browser_2511533020 {
    public static void main(String[] args) {

        Stack<Website_2511533020> history_3020 = new Stack<>();
        Scanner input_3020 = new Scanner(System.in);
        int pilihan_3020;

        System.out.println("=== Browser History NIM: 2511533020 ===\n");

        do {

            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History (isEmpty / size)");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan_3020 = input_3020.nextInt();
            input_3020.nextLine(); //

            switch (pilihan_3020) {
                case 1:
                    kunjungiWebsite(history_3020, input_3020);
                    break;
                case 2:
                    tombolBack(history_3020);
                    break;
                case 3:
                    lihatHalamanAktif(history_3020);
                    break;
                case 4:
                    cekStatusHistory(history_3020);
                    break;
                case 5:
                    System.out.println("Keluar dari program. Terima
kasih.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid! Silakan coba
lagi.\n");
            }
        } while (pilihan_3020 != 5);

        input_3020.close();
    }

```

```java
    public static void kunjungiWebsite(Stack<Website_2511533020> stack, Scanner sc) {
        System.out.print("Masukkan Judul: ");
        String judul = sc.nextLine();
        System.out.print("Masukkan URL: ");
        String url = sc.nextLine();

        Website_2511533020 halamanBaru = new Website_2511533020(judul, url);
        stack.push(halamanBaru);
        System.out.println("Berhasil mengunjungi halaman!\n");
    }


    public static void tombolBack(Stack<Website_2511533020> stack) {
        if (stack.isEmpty()) {
            System.out.println("Tidak ada riwayat. Stack kosong, tidak bisa back.\n");
        } else {
            Website_2511533020 halamanTerhapus = stack.pop();
            System.out.println("Kembali ke halaman sebelumnya. Halaman yang dihapus: "
                                + halamanTerhapus.getJudul_3020() + "\n");
        }
    }


    public static void lihatHalamanAktif(Stack<Website_2511533020> stack) {
        if (stack.isEmpty()) {
            System.out.println("Tidak ada halaman yang sedang aktif. History kosong.\n");
        } else {
            Website_2511533020 halamanAktif = stack.peek();
            System.out.println("Halaman aktif saat ini: " + halamanAktif.toString() + "\n");
        }
    }


    public static void cekStatusHistory(Stack<Website_2511533020> stack) {
        if (stack.isEmpty()) {
            System.out.println("Status History: Stack kosong (belum ada riwayat).\n");
        } else {
            System.out.println("Status History: Terdapat " + stack.size() + " halaman dalam riwayat.\n");
        }
    }
}
```

Kelas Browser_2511533020 merupakan kelas driver yang mengimplementasikan simulasi browser history menggunakan struktur data Stack bawaan Java (java.util.Stack). Di dalam method main, dibuat sebuah objek Stack<Website_2511533020> bernama history_3020 untuk

menyimpan riwayat halaman yang dikunjungi, serta objek Scanner untuk menerima input pengguna. Program menampilkan menu berulang dengan pilihan: Kunjungi Website (push), Tombol Back (pop), Lihat Halaman Aktif (peek), Cek Status History, dan Keluar. Setiap pilihan memanggil method statis yang sesuai. Method kunjungiWebsite() meminta input judul dan URL, membuat objek Website_2511533020, lalu memanggil push() untuk menambahkan ke stack. Method tombolBack() mengecek apakah stack kosong menggunakan isEmpty(); jika tidak kosong, ia memanggil pop() untuk menghapus halaman teratas dan menampilkan judul halaman yang dihapus.

Penanganan kondisi stack kosong dilakukan dengan pesan error yang ramah, sehingga program tidak mengalami EmptyStackException. Method lihatHalamanAktif() menggunakan peek() untuk menampilkan halaman teratas tanpa menghapusnya. Method cekStatusHistory() menampilkan jumlah total halaman dalam stack menggunakan size() atau pesan "kosong" jika isEmpty() bernilai true. Program ini memenuhi seluruh kriteria: penggunaan Stack dengan benar, aturan penamaan (kelas berakhiran NIM lengkap, atribut berakhiran 4 digit NIM), penanganan kondisi stack kosong, serta kerapihan kode dengan indentasi dan komentar.

- KELAS ADT

```java
package pekan3;

public class Website_2511533020 {
    // Atribut (private untuk enkapsulasi)
    private String judul_3020;
    private String url_3020;

    // Constructor
    public Website_2511533020(String judul_3020, String url_3020) {
        this.judul_3020 = judul_3020;
        this.url_3020 = url_3020;
    }

    // Selektor (Getter)
    public String getJudul_3020() {
        return judul_3020;
    }

    public String getUrl_3020() {
        return url_3020;
    }

    // Mutator (Setter)
    public void setJudul_3020(String judul_3020) {
        this.judul_3020 = judul_3020;
    }

```

```java
    public void setUrl_3020(String url_3020) {
        this.url_3020 = url_3020;
    }


    @Override
    public String toString() {
        return judul_3020 + " (" + url_3020 + ")";
    }
}

```

Kelas Website_2511533020 merupakan ADT (Abstract Data Type) yang merepresentasikan sebuah halaman website. Kelas ini memiliki dua atribut yaitu judul_3020 dan url_3020, keduanya bertipe String dan diberi akhiran 3020 sesuai aturan (4 digit terakhir NIM). Atribut-atribut tersebut bersifat private sehingga tidak dapat diakses langsung dari luar kelas, melainkan melalui method-method yang disediakan. Kelas ini dilengkapi dengan konstruktor berparameter untuk menginisialisasi kedua atribut saat objek dibuat.

Selain itu, tersedia selektor (getter) yaitu getJudul_3020() dan getUrl_3020() untuk mengambil nilai atribut, serta mutator (setter) yaitu setJudul_3020() dan setUrl_3020() untuk mengubah nilai atribut. Method toString() di-override agar objek website dapat dicetak langsung dengan format "Judul (URL)", yang memudahkan proses menampilkan data ke layar.

Ouput:

```
=== Browser History NIM: 2511533020 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (isEmpty / size)
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL:  https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!
```